

COMPTE RENDU

Administration Réseaux et Programmation Système - TP0 - Construction
d'un Réseau Virtuel

3e année Cybersécurité - École Supérieure d'Informatique et du
Numérique (ESIN)
Collège d'Ingénierie & d'Architecture (CIA)

Étudiant : HATHOUTI Mohammed Taha

Filière : Cybersécurité

Année : 2025/2026

Enseignant : M.Bakhouya & M.Aqachtoul

Date : 15 mars 2026

Table des matières

1	Partie 1 : Construction du réseau virtuel	2
1.1	Question 1 — Correspondance des rôles	2
1.2	Question 2 — Tableau de correspondance Linux	2
1.3	Question 3 — Pourquoi les namespaces sont-ils équivalents aux équipements Cisco ?	2
1.4	Question 4 — Tableaux de correspondance interfaces et adressage	2
1.5	Question 5 — Étapes de déploiement	3
2	Partie 2 : Tests de connectivité	5
2.1	Scénario 1 — Existence des namespaces	5
2.2	Scénario 2 — Placement des interfaces	5
2.3	Scénario 3 — Configuration IP	5
2.4	Scénario 4 — Tests de connectivité locale (PC ↔ Passerelle)	6
2.5	Scénario 5 — Test de routage inter-LAN (PC1 ↔ PC2)	6
2.6	Scénario 6 — Test du lien Passerelle ↔ FAI (WAN)	7
2.7	Scénario 7 — Test bout-en-bout PC → FAI (avant NAT)	7
3	Questions complémentaires	8
	Conclusion	10

1 Partie 1 : Construction du réseau virtuel

1.1 Question 1 — Correspondance des rôles

Dans ce lab Linux, chaque namespace réseau joue le rôle d'un équipement Cisco distinct :

- **PC Cisco** : namespaces PC1 et PC2
- **Routeur Passerelle Cisco** : namespace GATEWAY
- **Routeur FAI Cisco** : namespace ISP

1.2 Question 2 — Tableau de correspondance Linux

Composant Cisco	Équivalent Linux
PC	Namespace (PC1, PC2)
Routeur	Namespace réseau (GATEWAY, ISP)
Interface G0/0	Interface <code>veth</code>
Interface G0/1	Interface <code>veth</code>
Bloc IP public	Adresse IP sur <code>veth3</code>
Nuage Internet	Namespace ISP + loopback

1.3 Question 3 — Pourquoi les namespaces sont-ils équivalents aux équipements Cisco ?

Chaque namespace réseau possède ses propres interfaces, sa propre table de routage et ses propres règles de pare-feu/NAT. Cela est équivalent au fonctionnement indépendant de chaque routeur Cisco : deux namespaces distincts ne peuvent pas se voir ni s'influencer mutuellement sans configuration explicite de liens virtuels entre eux.

1.4 Question 4 — Tableaux de correspondance interfaces et adressage

Interfaces de la passerelle :

Type	Interface
Interfaces internes (LAN)	<code>gw1</code> , <code>gw2</code>
Interface externe (WAN)	<code>veth3</code>
Commande de vérification	<code>ip addr</code>

Correspondance Cisco / Linux :

Interface Cisco	Interface Linux
G0/0 (LAN1)	<code>gw1</code>
G0/0 (LAN2)	<code>gw2</code>
G0/1 (WAN)	<code>veth3</code>
FAI G0/0	<code>isp1</code>

1.5 Question 5 — Étapes de déploiement

Étape 1 : Configuration des namespaces réseau

Listing 1 – Création des namespaces

```
1 # Créer les namespaces
2 sudo ip netns add PC1
3 sudo ip netns add PC2
4 sudo ip netns add GATEWAY
5 sudo ip netns add ISP
6
7 # Créer les paires veth pour connecter les namespaces
8 sudo ip link add veth1 type veth peer name gw1
9 sudo ip link add veth2 type veth peer name gw2
10 sudo ip link add veth3 type veth peer name isp1
11
12 # Assigner les extrémités veth aux namespaces
13 sudo ip link set veth1 netns PC1
14 sudo ip link set veth2 netns PC2
15 sudo ip link set gw1 netns GATEWAY
16 sudo ip link set gw2 netns GATEWAY
17 sudo ip link set veth3 netns GATEWAY
18 sudo ip link set isp1 netns ISP
```

Listing 2 – Adressage IP et activation des interfaces

```
1 # PC1
2 sudo ip netns exec PC1 ip addr add 192.168.10.10/24 dev veth1
3 sudo ip netns exec PC1 ip link set veth1 up
4 sudo ip netns exec PC1 ip link set lo up
5
6 # PC2
7 sudo ip netns exec PC2 ip addr add 192.168.20.10/24 dev veth2
8 sudo ip netns exec PC2 ip link set veth2 up
9 sudo ip netns exec PC2 ip link set lo up
10
11 # GATEWAY (côté LAN)
12 sudo ip netns exec GATEWAY ip addr add 192.168.10.1/24 dev gw1
13 sudo ip netns exec GATEWAY ip addr add 192.168.20.1/24 dev gw2
14 sudo ip netns exec GATEWAY ip link set gw1 up
15 sudo ip netns exec GATEWAY ip link set gw2 up
16 sudo ip netns exec GATEWAY ip link set lo up
17
18 # GATEWAY (côté WAN)
19 sudo ip netns exec GATEWAY ip addr add 209.165.200.226/27 dev
    veth3
20 sudo ip netns exec GATEWAY ip link set veth3 up
21
22 # ISP
23 sudo ip netns exec ISP ip addr add 209.165.200.225/27 dev isp1
24 sudo ip netns exec ISP ip link set isp1 up
25 sudo ip netns exec ISP ip link set lo up
```

```

26
27 # Activer le loopback dans tous les namespaces
28 for ns in PC1 PC2 GATEWAY ISP; do
29     sudo ip netns exec $ns ip link set lo up
30 done

```

Étape 2 : Configuration du routage

Listing 3 – Routes par défaut et forwarding IP

```

1 # Routes par défaut pour les PCs
2 sudo ip netns exec PC1 ip route add default via 192.168.10.1
3 sudo ip netns exec PC2 ip route add default via 192.168.20.1
4
5 # Route par défaut pour la passerelle (vers le FAI)
6 sudo ip netns exec GATEWAY ip route add default via
    209.165.200.225
7
8 # Activation du forwarding IP sur GATEWAY
9 sudo ip netns exec GATEWAY sysctl -w net.ipv4.ip_forward=1

```

Étape 3 : Script Bash automatisé de configuration réseau

Le script `setup_network.sh` automatise entièrement le déploiement. Il inclut un mécanisme de nettoyage préventif, la création des namespaces, la création des liens virtuels, l'adressage IP, l'activation des interfaces et du loopback, ainsi que la configuration du routage.

Listing 4 – Extrait du script `setup_network.sh` — nettoyage préventif

```

1 for ns in PC1 PC2 GATEWAY ISP; do
2     sudo ip netns list | grep -q "^${ns}" \
3     && sudo ip netns del "$ns" 2>/dev/null
4 done
5 for lien in veth1 veth2 veth3; do
6     sudo ip link show "$lien" &>/dev/null \
7     && sudo ip link del "$lien" 2>/dev/null
8 done

```

Étape 4 : Script Bash de nettoyage complet

Le script `cleanup_network.sh` supprime tous les namespaces et liens veth créés, afin de permettre une reconstruction propre de l'environnement sans conflits.

Listing 5 – Extrait du script `cleanup_network.sh`

```

1 for ns in PC1 PC2 GATEWAY ISP; do
2     if ip netns list | grep -q "^${ns}"; then
3         sudo ip netns del "$ns"
4     fi
5 done
6 for lien in veth1 veth2 veth3 gw1 gw2 isp1; do

```

```

7   if ip link show "$lien" &>/dev/null; then
8       sudo ip link del "$lien" 2>/dev/null
9   fi
10 done

```

2 Partie 2 : Tests de connectivité

2.1 Scénario 1 — Existence des namespaces

Listing 6 – Vérification des namespaces créés

```

1 $ ip netns list
2 ISP (id: 3)
3 GATEWAY (id: 2)
4 PC2 (id: 1)
5 PC1 (id: 0)

```

Les quatre namespaces sont bien présents et identifiés par des IDs distincts.

2.2 Scénario 2 — Placement des interfaces

Listing 7 – Vérification du placement des interfaces

```

1 $ sudo ip netns exec PC1 ip link
2 1: lo: <LOOPBACK,UP,LOWER_UP> ...
3 67: veth1@if66: <BROADCAST,MULTICAST,UP,LOWER_UP> ... link-netns
   GATEWAY
4
5 $ sudo ip netns exec GATEWAY ip link
6 1: lo: <LOOPBACK,UP,LOWER_UP> ...
7 66: gw1@if67: <BROADCAST,MULTICAST,UP,LOWER_UP> ... link-netns
   PC1
8 68: gw2@if69: <BROADCAST,MULTICAST,UP,LOWER_UP> ... link-netns
   PC2
9 71: veth3@if70: <BROADCAST,MULTICAST,UP,LOWER_UP> ... link-netns
   ISP
10
11 $ sudo ip netns exec ISP ip link
12 1: lo: <LOOPBACK,UP,LOWER_UP> ...
13 70: isp1@if71: <BROADCAST,MULTICAST,UP,LOWER_UP> ... link-netns
   GATEWAY

```

Les résultats confirment : PC1 → veth1; PC2 → veth2; GATEWAY → gw1, gw2, veth3; ISP → isp1.

2.3 Scénario 3 — Configuration IP

Listing 8 – Vérification de l'adressage IP

```

1 $ sudo ip netns exec PC1 ip addr show veth1

```

```

2      inet 192.168.10.10/24 scope global veth1
3
4 $ sudo ip netns exec PC2 ip addr show veth2
5      inet 192.168.20.10/24 scope global veth2
6
7 $ sudo ip netns exec GATEWAY ip addr show gw1
8      inet 192.168.10.1/24 scope global gw1
9
10 $ sudo ip netns exec GATEWAY ip addr show gw2
11      inet 192.168.20.1/24 scope global gw2
12
13 $ sudo ip netns exec GATEWAY ip addr show veth3
14      inet 209.165.200.226/27 scope global veth3
15
16 $ sudo ip netns exec ISP ip addr show isp1
17      inet 209.165.200.225/27 scope global isp1

```

2.4 Scénario 4 — Tests de connectivité locale (PC ↔ Passerelle)

Listing 9 – Ping PC1 vers passerelle LAN1

```

1 $ sudo ip netns exec PC1 ping -c 4 192.168.10.1
2 PING 192.168.10.1 (192.168.10.1) 56(84) bytes of data.
3 64 bytes from 192.168.10.1: icmp_seq=1 ttl=64 time=0.066 ms
4 64 bytes from 192.168.10.1: icmp_seq=2 ttl=64 time=0.076 ms
5 64 bytes from 192.168.10.1: icmp_seq=3 ttl=64 time=0.045 ms
6 64 bytes from 192.168.10.1: icmp_seq=4 ttl=64 time=0.052 ms
7 --- 192.168.10.1 ping statistics ---
8 4 packets transmitted, 4 received, 0% packet loss, time 3072ms

```

Listing 10 – Ping PC2 vers passerelle LAN2

```

1 $ sudo ip netns exec PC2 ping -c 4 192.168.20.1
2 PING 192.168.20.1 (192.168.20.1) 56(84) bytes of data.
3 64 bytes from 192.168.20.1: icmp_seq=1 ttl=64 time=0.244 ms
4 64 bytes from 192.168.20.1: icmp_seq=2 ttl=64 time=0.053 ms
5 64 bytes from 192.168.20.1: icmp_seq=3 ttl=64 time=0.069 ms
6 64 bytes from 192.168.20.1: icmp_seq=4 ttl=64 time=0.067 ms
7 --- 192.168.20.1 ping statistics ---
8 4 packets transmitted, 4 received, 0% packet loss, time 3100ms

```

Les deux pings aboutissent avec 0% de perte, confirmant la connectivité locale dans chaque LAN.

2.5 Scénario 5 — Test de routage inter-LAN (PC1 ↔ PC2)

Listing 11 – Ping PC1 vers PC2 via GATEWAY

```

1 $ sudo ip netns exec PC1 ping -c 4 192.168.20.10
2 PING 192.168.20.10 (192.168.20.10) 56(84) bytes of data.
3 64 bytes from 192.168.20.10: icmp_seq=1 ttl=63 time=0.094 ms

```

```

4 64 bytes from 192.168.20.10: icmp_seq=2 ttl=63 time=0.088 ms
5 64 bytes from 192.168.20.10: icmp_seq=3 ttl=63 time=0.102 ms
6 64 bytes from 192.168.20.10: icmp_seq=4 ttl=63 time=0.119 ms
7 --- 192.168.20.10 ping statistics ---
8 4 packets transmitted, 4 received, 0% packet loss, time 3079ms

```

Listing 12 – Ping PC2 vers PC1 via GATEWAY

```

1 $ sudo ip netns exec PC2 ping -c 4 192.168.10.10
2 PING 192.168.10.10 (192.168.10.10) 56(84) bytes of data.
3 64 bytes from 192.168.10.10: icmp_seq=1 ttl=63 time=0.410 ms
4 64 bytes from 192.168.10.10: icmp_seq=2 ttl=64 time=0.065 ms
5 64 bytes from 192.168.10.10: icmp_seq=3 ttl=63 time=0.105 ms
6 64 bytes from 192.168.10.10: icmp_seq=4 ttl=63 time=0.090 ms
7 --- 192.168.10.10 ping statistics ---
8 4 packets transmitted, 4 received, 0% packet loss, time 3085ms

```

Le `ttl=63` (au lieu de 64) confirme que les paquets ont bien traversé un routeur (GATEWAY), validant le forwarding IP inter-LAN.

2.6 Scénario 6 — Test du lien Passerelle ↔ FAI (WAN)

Listing 13 – Ping GATEWAY vers ISP et retour

```

1 $ sudo ip netns exec GATEWAY ping -c 4 209.165.200.225
2 PING 209.165.200.225 (209.165.200.225) 56(84) bytes of data.
3 64 bytes from 209.165.200.225: icmp_seq=1 ttl=64 time=0.111 ms
4 64 bytes from 209.165.200.225: icmp_seq=2 ttl=64 time=0.072 ms
5 64 bytes from 209.165.200.225: icmp_seq=3 ttl=64 time=0.054 ms
6 64 bytes from 209.165.200.225: icmp_seq=4 ttl=64 time=0.050 ms
7 --- 209.165.200.225 ping statistics ---
8 4 packets transmitted, 4 received, 0% packet loss, time 3105ms
9
10 $ sudo ip netns exec ISP ping -c 4 209.165.200.226
11 PING 209.165.200.226 (209.165.200.226) 56(84) bytes of data.
12 64 bytes from 209.165.200.226: icmp_seq=1 ttl=64 time=0.108 ms
13 64 bytes from 209.165.200.226: icmp_seq=2 ttl=64 time=0.050 ms
14 64 bytes from 209.165.200.226: icmp_seq=3 ttl=64 time=0.052 ms
15 64 bytes from 209.165.200.226: icmp_seq=4 ttl=64 time=0.059 ms
16 --- 209.165.200.226 ping statistics ---
17 4 packets transmitted, 4 received, 0% packet loss, time 3067ms

```

Le lien WAN entre GATEWAY et ISP est opérationnel dans les deux sens.

2.7 Scénario 7 — Test bout-en-bout PC → FAI (avant NAT)

Listing 14 – Tentative de ping PC1 vers ISP sans NAT

```

1 $ sudo ip netns exec PC1 ping -c 4 209.165.200.225
2 ping: connect: Network is unreachable

```


Ce résultat est **attendu et correct**. Le paquet quitte PC1 avec la source 192.168.10.10, arrive à GATEWAY qui le transfère vers ISP. ISP reçoit le paquet mais **ne connaît pas la route retour** vers 192.168.10.0/24 : l'Echo Reply est abandonné.

Pour confirmer que le paquet arrive bien jusqu'à ISP, on peut utiliser `tcpdump` :

Listing 15 – Preuve de connectivité unidirectionnelle avec `tcpdump`

```

1 # Terminal 1      capture sur ISP
2 $ sudo ip netns exec ISP tcpdump -i isp1 icmp
3
4 # Terminal 2      ping depuis PC1
5 $ sudo ip netns exec PC1 ping -c 2 209.165.200.225
6
7 # Résultat tcpdump : on voit les Echo Request mais PAS les Echo
  Reply
8 # => connectivité unidirectionnelle confirmée

```

Analyse de la trace paquet :

1. PC1 envoie : src=192.168.10.10, dst=209.165.200.225 via sa route par défaut (192.168.10.1)
2. GATEWAY reçoit, consulte sa table de routage, transfère via `veth3` vers ISP
3. ISP reçoit l'Echo Request mais sa table de routage ne contient pas 192.168.10.0/24
4. L'Echo Reply est abandonné — **le routage doit fonctionner dans les deux sens**

Ce comportement démontre la nécessité du **NAT** : en traduisant l'adresse source privée 192.168.10.10 en adresse publique 209.165.200.226, ISP pourra répondre correctement.

3 Questions complémentaires

Q1 — Quels namespaces ont été créés ?

PC1, PC2, GATEWAY, ISP. Chacun représente un nœud réseau indépendant : PC1 et PC2 sont les hôtes des LANs, GATEWAY est le routeur central, ISP simule le réseau public.

Q2 — Quelles interfaces possède chaque namespace ?

PC1 : `veth1` / PC2 : `veth2` / GATEWAY : `gw1`, `gw2`, `veth3` / ISP : `isp1`

Q3 — Adresses IP assignées

Namespace	Interface	Adresse IP
PC1	<code>veth1</code>	192.168.10.10/24
PC2	<code>veth2</code>	192.168.20.10/24
GATEWAY	<code>gw1</code>	192.168.10.1/24
GATEWAY	<code>gw2</code>	192.168.20.1/24
GATEWAY	<code>veth3</code>	209.165.200.226/27
ISP	<code>isp1</code>	209.165.200.225/27

Q4 — Quel équipement joue le rôle de routeur ?

Le namespace `GATEWAY`, car il connecte trois réseaux distincts (LAN1, LAN2, WAN) et transfère les paquets entre eux grâce au forwarding IP activé (`net.ipv4.ip_forward=1`).

Q5 — Pourquoi PC1 peut pinguer la passerelle mais pas le FAI directement ?

PC1 est dans `192.168.10.0/24` et la passerelle est son prochain saut direct — elle est accessible. Le FAI est dans `209.165.200.224/27` : le paquet peut y parvenir via `GATEWAY`, mais ISP ne connaît pas `192.168.10.0/24` et ne peut pas renvoyer l'Echo Reply. Sans NAT, l'IP source privée est non routable sur le réseau public.

Q6 — Route par défaut de PC1 et PC2

`PC1: default via 192.168.10.1` — `PC2: default via 192.168.20.1`. Ces routes indiquent à chaque hôte d'envoyer tout trafic inconnu vers la passerelle.

Q7 — Route par défaut de la passerelle ? Pourquoi ?

`default via 209.165.200.225` : elle pointe vers le FAI pour tout trafic destiné à l'extérieur des LANs. Sans cette route, la passerelle ne saurait pas vers où acheminer les paquets à destination d'Internet.

Q8 — Que se passe-t-il quand PC1 envoie un paquet à 209.165.200.225 ?

Le paquet quitte `veth1` vers `gw1` (`GATEWAY`). `GATEWAY` consulte sa table de routage et transfère via `veth3` vers `isp1`. ISP reçoit l'Echo Request mais voit une IP source privée `192.168.10.10` pour laquelle il n'a pas de route retour — la réponse est abandonnée.

Q9 — Effet de `sysctl -w net.ipv4.ip_forward=1`

Active le transfert de paquets IP entre les interfaces du namespace `GATEWAY`, le transformant en routeur capable de relayer les paquets entre `gw1`, `gw2` et `veth3`.

Q10 — Que se passerait-il si le forwarding était désactivé ?

`GATEWAY` ne transmettrait plus les paquets entre ses interfaces. PC1 ne pourrait atteindre ni PC2 ni le FAI — tous les paquets inter-réseaux seraient silencieusement abandonnés.

Q11 / Q12 — Le NAT existe-t-il à ce stade ?

Non. Seul le forwarding IP est activé. Les paquets conservent leur IP source privée (`192.168.x.x`), qui n'est pas routable sur le réseau public.

Q13 — Quel problème le NAT résoudra-t-il ?

Le NAT traduira les IPs privées LAN en l'IP publique de la passerelle (209.165.200.226), permettant au FAI de répondre correctement. La communication Internet devient possible sans que le FAI n'ait à connaître les sous-réseaux privés.

Q14 — Ce qu'on peut pinguer sans NAT

Listing 16 – Tests de connectivité sans NAT

```
1 # Fonctionne : même sous-réseau (LAN local)
2 sudo ip netns exec PC1 ping -c 4 192.168.10.1 # PC1 ->
   Passerelle LAN1
3 sudo ip netns exec PC2 ping -c 4 192.168.20.1 # PC2 ->
   Passerelle LAN2
4
5 # Fonctionne : routage inter-LAN privé via GATEWAY (forwarding
   activé)
6 sudo ip netns exec PC1 ping -c 4 192.168.20.10 # PC1 -> PC2
7 sudo ip netns exec PC2 ping -c 4 192.168.10.10 # PC2 -> PC1
8
9 # Echoue : IP source privée non routable publiquement, ISP ne
   peut pas répondre
10 sudo ip netns exec PC1 ping -c 4 209.165.200.225 # PC1 -> ISP
11 sudo ip netns exec PC2 ping -c 4 209.165.200.225 # PC2 -> ISP
```

Les pings inter-LAN fonctionnent car le routage reste dans la sphère privée (privé → privé). Les pings vers le FAI échouent car l'ISP voit une adresse source privée 192.168.x.x pour laquelle il n'a aucune route retour.

Conclusion

Ce TP0 a permis de construire un réseau virtuel complet sur une seule machine Linux, sans aucun matériel physique, en tirant parti des namespaces réseau, des interfaces veth et de l'outil `iproute2`. La topologie réalisée reproduit fidèlement un scénario réel d'entreprise : deux LANs privés connectés via un routeur passerelle à un réseau FAI simulé.

Les tests de connectivité ont validé le fonctionnement correct de chaque couche : la connectivité locale au sein de chaque LAN (Scénario 4), le routage inter-LAN rendu possible par le forwarding IP de GATEWAY (Scénario 5), et la liaison WAN entre la passerelle et le FAI (Scénario 6). Le Scénario 7 a démontré de manière concrète la règle fondamentale du routage : **le chemin de retour est aussi important que le chemin aller**. L'échec du ping PC1 → ISP (sans routes retour sur ISP) illustre précisément pourquoi le NAT est indispensable dans les réseaux utilisant des adresses privées.

La production de trois scripts Bash (`setup_network.sh`, `test_connectivite.sh`, `cleanup_network.sh`) a permis d'automatiser l'ensemble du cycle de vie du lab, garantissant la reproductibilité des expériences et évitant les conflits de configuration entre les sessions.